Lab 1 - Profiling Parallel Code (1st part)

Advanced Computer Architectures

University of Minho

The Lab 1 focus on the development of efficient CPU code by revisiting the programming principles that have a relevant impact on performance, such as vectorisation and scalability of multi-threaded algorithms. Submit jobs in Deucalion arm partitions to measure code execution times.

This lab tutorial includes two parts, a simple tutorial for connecting to Deucalion and some usefull commands and a second part that includes four exercises that should be solved in order (Ext 1.1, 1.2, 1.3, 1.4).

1 The Deucalion super-computer

In this section, you will lear how to:

- Log in to Deucalion;
- Load software modules;
- Create/edit files remotely;
- Compile programs;
- Submit jobs.

If this is your first contact with HPC systems, take your time to follow each step carefully. By the end of this tutorial, you will be able to run your first program in Deucalion and manage your files easily.

1.1 Prerequisites

Before starting:

- Make sure you have an account in Deucalion;
- Have a ssh client installed on your machine (e.g. Linux has ssh preinstalled/windows can use PuTTY);
- Have a SSH key;

Usefull documentation can be found at:

`https://docs.deucalion.macc.fccn.pt/start/`

1.2 Logging into Deucalion

The Deucalion super-computer is a set of powerfull servers you can access remotely. By now you should have already installed a ssh client and have your ssh key. To connect to Deucalion do:

```
ssh -i <path-to-private-key> <username>@login.deucalion.macc.fccn.pt
```

You should see a welcoming message now.

1.3 Understanding the Deucalion Environment

After successfully logging in, you are now present in what is called a "Login Node". This is a node that the super-computer makes available for users to interact with it. This is a shared environment and you will be sharing it with other users, thus, NEVER run heavy computations here! To run your software you should use the

```
sbatch
```

command that will be presented latter. Useful documentation:

```
https://docs.deucalion.macc.fccn.pt/deucalion/
```

1.4 Loading software Modules

At this point, the command line you are using is executing commands remotely. This means that you no longer have access to software that you usually use in your personal computer. Fortunately, Deucalion has most of the software that you will ever need available to use. It uses the module system to manage software.

- Check available modules: `module avail`;
- Load module: `module load GCC/X.Y.Z`;
- Check loaded modules: `module list`;
- Remove loaded module: `module unload GCC/X.Y.Z`

1.5 Creating and Editing files

You can create and edit files directly using editors like `nano`. To save and exit use:

- Press `CTRL+O` to save;
- Press: `CTRL+X` to exit;

1.6 Submitting Jobs

As mentioned earlier, when you log in, you are connected to a login node. This node acts as a front-end interface to Deucalion and is intended only for lightweight tasks such as editing files, compiling light code or submitting jobs. To run your software on the actual compute dedicated hardware of Deucalion, a scheduler is used to manage and allocate resources for the users. Here is a simple script you can use to submit a job.

```
#!/bin/bash

#SBATCH --nodes=1
#SBATCH --exclusive
#SBATCH --time=00:05:00
#SBATCH --partition=normal-arm
#SBATCH --account=f... # check the board

module load GCC/11.3.0

echo "Compiling the code"
echo ""

gcc my_program.c -o my_code

echo ""
echo "Running the code"
echo ""

./my_code

echo ""
echo "Finished"
```

2 Social media network

Goals: to develop skills in the fundamentals of thread programming.

Imagine you work at a company running a large social network (think X, LinkdIn, or TikTok). Each user in the network has a small feature vector (e.g. age, activity level, etc). The connections between users can be represented as a matrix with 0s and 1s. Most users have only a few connections. A few "celebrities" have **millions**. The team responsible for the algorithm that recommends *friends/followers* asks you for data they need to improve it. They want: *For every user, the average feature vector of its friends.* After studying the problem you arrived to a simple solution. The needed data can be obtained by simply multiplying the matrix of friendships with the matrix of user features and apply the average. E.g:

$$\mathbf{Followers}_{3x3} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

$$\mathbf{User_features}_{3x2} = \begin{bmatrix} 0.2 & 0.4 \\ 0.8 & 0.1 \\ 0.3 & 0.9 \end{bmatrix}$$

$$\mathbf{User_followers_features}_{3x2} = \mathbf{Followers}_{3x3} \times \mathbf{User_features}_{3x2}$$

$$\begin{aligned} \mathbf{Followers} \times \mathbf{User_features} &= \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0.2 & 0.4 \\ 0.8 & 0.1 \\ 0.3 & 0.9 \end{bmatrix} \\ &= \begin{bmatrix} 0.8 & 0.1 \\ 0.2 + 0.3 & 0.4 + 0.9 \\ 0.8 & 0.1 \end{bmatrix} \\ &= \begin{bmatrix} 0.8 & 0.1 \\ 0.5 & 1.3 \\ 0.8 & 0.1 \end{bmatrix} \end{aligned}$$

If we average each row over the number of follows:

$$\mathbf{X}_{\text{Followers}} = \begin{bmatrix} 0.8 & 0.1 \\ 0.25 & 0.65 \\ 0.8 & 0.1 \end{bmatrix}$$

The code for this exercise can be found in:

`/projects/F202500010HPCVLABUMINHI/ACA/lab1`

Copy it into your directory.

2.1 Exercises

Take a moment to look at the code in src/social_media.cpp.

Lab / Ext 1.1 Complete the worker algorithm to answer the question asked.

Lab / Ext 1.2 Create a function that follows a master-worker schema to parallelize the code and distributes the workload equally among available threads in the system. The parallelized code should deliver the same result as the original sequential multiplication.

Lab / Ext 1.3 Create two executables resultant from different compilation options:

```
gcc -Wall -O2 -g -o social_media_O2
gcc -Wall -O3 -g -o social_media_O3
```

Make the necessary changes in the Makefile.

Use *perf* to profile both executables for a dataset size that does not fit in cache. Do not forget to load the necessary modules.

Profile both executables sing *perf* for a single thread (sequential) and 24 threads. Choose the hotsposts profiling to get an overall idea of the code bottlenecks.

```
perf stat ./executable
```

Lab / Ext 1.4 Extend the code to ensure that each thread writes on a private chunk of memory (the output matrix), and implement a way to efficiently concatenate the results into a single output matrix after each thread processes its data. Compare the execution times of this version of the code with **Lab / Ext 1.3** for problem sizes that do not fit in cache. Does the performance scale as expected? Profile the code and identify the inefficiencies (CPU, memory, coding inefficiencies), and possibly their causes.